

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»



Инженерная школа информационных технологий и робототехники

Отделение информационных технологий

09.04.01 «Информатика и вычислительная техника»

Отчет по лабораторной работе №3
«Классификация объектов на изображении с помощью предобученной
нейросети VGG16»

МЕТОДЫ РАСПОЗНАВАНИЯ ОБРАЗОВ

Исполнитель:

студент группы

8BM22

Ямкин Н.Н.

Руководитель:

Доцент (ОИТ, ИШИТР)

Друки А.А.

Содержание

1. Цель работы	3
2. Задачи	3
3. Ход работы.....	4
4. Заключение	6

1. Цель работы

Получить навыки применения предобученной нейросети VGG16 для классификации объектов на изображениях.

2. Задачи

1. Импортировать предобученную нейросеть VGG16;
2. Использовать нейросеть VGG16 для классификации объектов на изображении;
3. Реализовать наложение шума на изображение;
4. Проанализировать как нейронная сеть будет классифицировать зашумленное изображение и на каком этапе начнет ошибаться в классификации.

VGG16 – модель сверточной нейронной сети, предложенная К. Simonyan и А. Zisserman из Оксфордского университета в статье “Very Deep Convolutional Networks for Large-Scale Image Recognition”. Модель достигает точности 92.7% — топ-5, при тестировании на ImageNet в задаче распознавания объектов на изображении. Этот датасет состоит из более чем 14 миллионов изображений, принадлежащих к 1000 классам.

VGG16 – одна из самых знаменитых моделей, отправленных на соревнование ILSVRC-2014. Она является улучшенной версией AlexNet, в которой заменены большие фильтры (размера 11 и 5 в первом и втором сверточном слое, соответственно) на несколько фильтров размера 3x3, следующих один за другим. Сеть VGG16 обучалась на протяжении нескольких недель при использовании видеокарт NVIDIA TITAN BLACK.

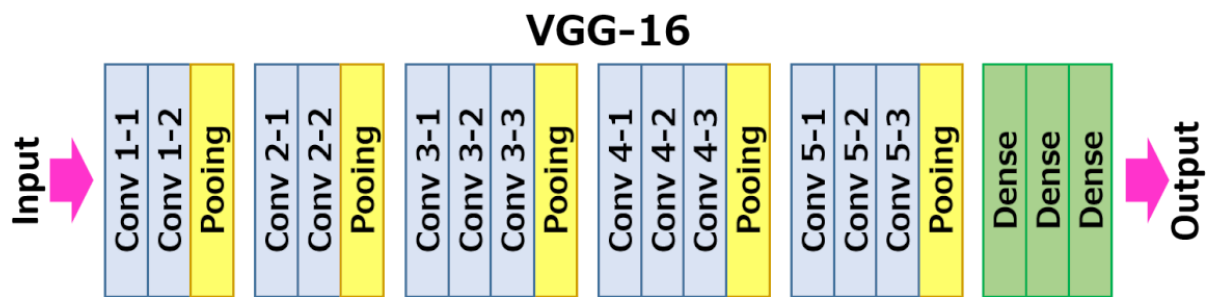


Рисунок 1 – Архитектура сети VGG16

3. Ход работы

Создадим новый блокнот в Google Colab и импортируем необходимые для работы с изображениями библиотеки:

```
import tensorflow as tf
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.vgg16 import preprocess_input,
decode_predictions
import numpy as np
import matplotlib.pyplot as plt
from foolbox import TensorFlowModel, Model
from foolbox.attacks import LinfFastGradientAttack
```

Определить модель нейросети, предобученной на ImageNet

```
model = VGG16(weights='imagenet')
```

Классификация объекта на изображении.

```
img_path = '/content/drive/MyDrive/amurskiy_tigr.jpg'
img = image.load_img(img_path, target_size=(224, 224))
x = np.array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
preds = model.predict(x)
answer = decode_predictions(preds, top=3)
for i in range(3):
    print('Класс объекта:', answer[0][i][1:][0], '-',
answer[0][i][1:][1]*100)
plt.imshow(img)
pass
```

```

1/1 [=====] - 1s 1s/step
Downloading data from https://storage.googleapis.com/download-obj441339002/tiger1.jpg
35363/35363 [=====] - 0s 0us/step
Класс объекта: tiger - 90.44018983840942
Класс объекта: tiger_cat - 9.513019025325775
Класс объекта: lynx - 0.029789001564495265

```

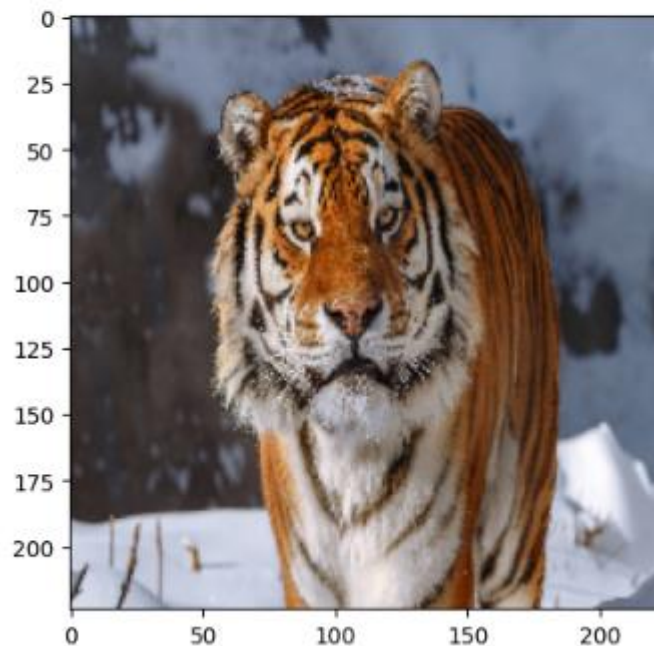


Рисунок 2 – Результат классификации

Наложим шум на изображение.

```

# Реализовать наложение шума на изображение
pre = dict(flip_axis=-1, mean=[103.939, 116.779, 123.68]) # RGB to BGR
fmodel: Model = TensorFlowModel(model, bounds=(0, 255), preprocessing=pre)

# apply the attack
attack = LinfFastGradientAttack()

#так как преобразования к изображению применяются внутри модели foolbox,
еще раз поместим его в переменную x
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
banana_image, banana_label = tf.convert_to_tensor(x),
tf.convert_to_tensor(tf.cast([954], tf.int64))

#Зададим значения допустимых возмущений
epsilons = [0.0, 0.001, 0.01, 0.03, 0.1, 0.3, 0.5, 1.0, 2.0, 5.0, 10.0]

raw_advs, clipped_advs, success = attack(fmodel, banana_image,
banana_label, epsilons=epsilons)

```

Проанализируем результаты классификации на зашумленном изображении.

```
# Проанализировать как нейронная сеть будет классифицировать зашумленное
изображение и на каком этапе начнет ошибаться в классификации
for sample in raw_advs:
    preds = model.predict(sample)
    print('Predicted:', decode_predictions(preds, top=3)[0])
    plt.figure()
    plt.imshow(sample[0] / 255)
    plt.title('Adversarial image')
    plt.show()
```

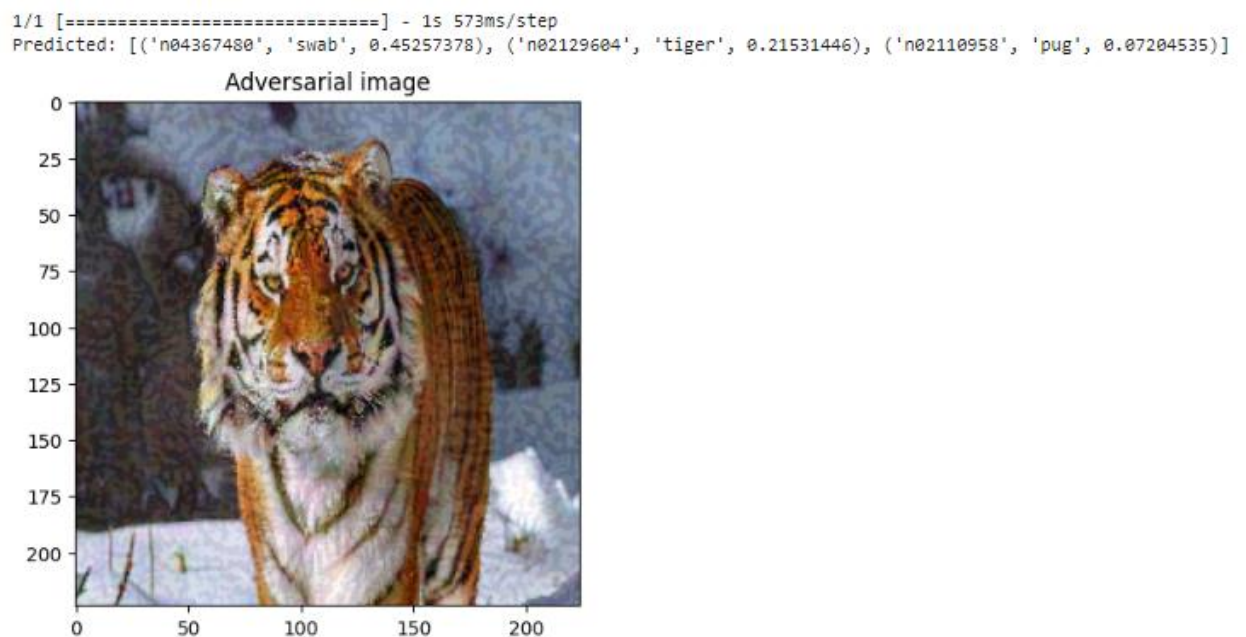


Рисунок 3 – Результат классификации при шуме 10

4. Заключение

В данной лабораторной работе была использована обученная модель VGG16 для предсказания классификации изображения. Модель была обучена на наборе данных ImageNet. Получена оценка вероятности того, к какой категории относится изображение.